❖ What Is Observability?

- **Observability** means:
  - Understanding what is happening inside your system (apps, servers, Kubernetes, cloud, etc.) by looking at the data it produces.
- Instead of guessing when something breaks, observability helps you **see the system's health in real time**.
- It uses **telemetry data**, which includes:
  - Metrics
  - Logs
  - Traces
  - (New) Profiles
    - These are called the **pillars of observability**.

❖ The 3 (Now 4) Pillars of Observability

➢ Metrics – "What is happening?"

- Metrics are **numbers collected over time**.
- Examples:
  - CPU usage
  - Memory usage
  - Request count
  - Response time
  - Error rate
- They help answer:
- Is the system healthy? Is performance normal?
- Metrics are fast, lightweight, and used for monitoring dashboards.

➢ Logs – “Why did it happen?”

- Logs are **detailed event records with timestamps**.
- Examples:
- Error messages
- Login attempts
- Database failures
- Configuration changes
- They help answer:
- Why did this error happen?
- Logs are very detailed but can be large and messy.

➢ Traces – “Where is the problem?”

- Traces show the **full journey of a request**.
- Example:

  User → API Gateway → Backend → Database → Response
- If something is slow, traces show:
- Which service caused the delay?
- This is very important in:
- Microservices
- Kubernetes
- Distributed system

➢ Profiles (New Pillar) – “Which line of code is slow?”

- Profiles go deeper into:
- CPU usage per function
- Memory allocation

  - Code-level bottlenecks

Profiles help developers optimize code performance.

Modern standards like OpenTelemetry are starting to support profiling too.

Types of Observability Metrics

There are 3 main types:

---

1. Application Metrics

These are related to your app performance.

Examples:

- Response time
- Request rate
- Throughput
- Error count

Used in APM (Application Performance Monitoring).

Helps developers monitor app health.

---

2. System (Infrastructure) Metrics

These show hardware and platform health.

Examples:

- CPU utilization
- Memory usage
- Disk I/O
- Network traffic

- Container resource usage

Very important in:

- Cloud
- VMs
- Docker
- Kubernetes

Helps DevOps teams manage infrastructure.

---

3. Business Metrics

These connect system performance to business results.

Examples:

- Conversion rate
- Revenue
- User retention
- Transactions per minute

Helps management understand business impact.

The 4 Golden Signals (Very Important for SRE)

In Site Reliability Engineering (SRE), there are **4 most important metrics**.

These are called the **Golden Signals**:

❖ Latency

- How long a request takes.
- Example:
- API response time = 200ms □
- API response time = 5 seconds □
- High latency = slow user experience.

❖ Traffic

- How many requests are coming in.
- Example:
- 100 requests/sec
- 10,000 requests/sec
- Helps plan scaling

❖ Errors

- How many requests failed.
- Example:
- 0.1% error rate □
- 10% error rate □
- High error rate = serious issue.

❖ Saturation

- How close your system is to maximum capacity.
- Examples:
- CPU at 95%
- Memory at 90%
- Disk full
- High saturation = system may crash soon.

If you monitor these 4 properly, you can detect most production issues early.

Best Practices for Observability

1. Define Clear Goals

Don’t monitor everything.

Ask:

- What matters to my business?
- What affects users?
- What can break production?

Monitor only important signals.

---

2. Use Open Standards

Avoid vendor lock-in.

Use open frameworks like:

- OpenTelemetry

It allows collecting:

- Metrics
- Logs
- Traces

From any system.

---

3. Automate Everything

- Automatic metric collection

- Automatic alerts
- Auto-scaling
- Auto dashboards

This reduces manual work.

---

4. Customize Dashboards

Default dashboards are not enough.

You should:

- Create dashboards based on your app
- Focus on golden signals
- Add alerts for critical thresholds

Why Observability Is Important

Without observability:

- You react after system crashes
- You guess the root cause
- You waste time debugging

With observability:

- You detect issues early
- You reduce downtime
- You improve performance
- You improve user experience
- You save cost

What Is Elastic Observability?

Elastic provides a tool called **Elastic Observability**.

It allows you to:

- Collect metrics, logs, traces
- Store and search them
- Visualize dashboards
- Detect anomalies
- Prevent outages

It works well with:

- Cloud
- Kubernetes
- Microservices
- OpenTelemetry

Simple Real-Life Example

Imagine you have a food delivery app running on Kubernetes.

One day users complain:
"App is very slow."

With observability:

1. Metrics show → Latency increased
2. Traffic shows → High user load
3. Saturation shows → CPU at 95%
4. Traces show → Database query slow
5. Logs show → DB connection timeout

Now you know:
The database is overloaded.

Without observability:

You would just restart pods and hope it works

Final Simple Summary

Observability = Understanding your system using data.

It uses:

- Metrics → What is happening
- Logs → Why it happened
- Traces → Where it happened
- Profiles → Which code caused it

Golden Signals:

- Latency
- Traffic
- Errors
- Saturation

Best practices:

- Monitor what matters
- Use open standards
- Automate
- Customize dashboards